Eight tools and one dispatcher

Animated explainer · src/tools.js

Tools are how the model acts on the world. tools.js defines the schemas the model sees, validates what comes back, dispatches to a handler, and returns a string for the model to read. Every error is an observation, never a crash.

9 scenes · 8 tools · 8 hard-deny patterns · 6000-char output cap

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · Schemas 4

03 · Validation 5

04 · Handlers 6

05 · Command policy 7

06 · Write tools 8

07 · Dispatch 9

08 · The bounds 10

09 · Recap 11

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

The model has eight ways to act.

list_files — Walk a folder tree.

read_file — Read with line numbers.

create_file — Create a new file.

write_file — Create or overwrite.

edit_file — Replace one exact snippet.

run_command — Run a real shell command.

remember — Save a durable note.

recall — Search earlier memories.

> Six tools change the workspace. Two change what the next process knows about it.
> **On screen:** The model sees eight tools. Six act on files and commands; two act on memory. The last two are special: remember and recall are the only effects that outlive the run. Everything the model can do is described here, in the schemas it receives.

02 · Schemas

Tool descriptions are prompt text.

SRC/TOOLS.JS · *source*

```
export const TOOL_SCHEMAS = [
  {
    type: 'function',
    function: {
      name: 'edit_file',
      description: 'Replace one exact snippet in an existing file. old_string must appear exactly once unless
replace_all is true.',
      parameters: {
        type: 'object',
        properties: {
          path: { type: 'string', description: 'File path relative to the workspace root' },
          old_string: { type: 'string', description: 'Exact text to replace, including indentation' },
          new_string: { type: 'string', description: 'Replacement text' },
          replace_all: { type: 'boolean', description: 'Replace every occurrence. Default false' },
        },
        required: ['path', 'old_string', 'new_string'],
        additionalProperties: false,
      },
    },
  },
];
```

The model chooses tools from these words. A vague description is a bad tool, no matter how good the handler is.

On screen: The schemas are not just validation. They are the model's instruction manual. Each description says when to use the tool and what an argument means. That is why the descriptions are carefully written. They are prompt text.

03 · Validation

Parse arguments before you trust them.

SRC/TOOLS.JS · *source*

```
function validate(name, raw) {
  const spec = PARAMS[name];
  let args;
  try {
    args = typeof raw === 'string' ? (raw.trim() ? JSON.parse(raw) : {}) : (raw ?? {});
  } catch (err) {
    return { error: 'ERROR: could not parse arguments for ' + name + ': ' + err.message };
  }
  if (args === null || typeof args !== 'object' || Array.isArray(args)) {
    return { error: 'ERROR: arguments for ' + name + ' must be a JSON object' };
  }
  for (const [key, type] of Object.entries(spec)) {
    // ...check the declared type...
  }
  return { args };
}
```

- 📄 JSON first a provider may send arguments as a string.
- 📦 Object only arrays and null are rejected.
- 🔤 Type check every declared parameter is checked.

> **On screen:** A raw tool call may contain a JSON string, an object, or nonsense. validate parses the string, requires an object, and checks the declared types. Bad arguments become an ERROR string. The handler is never called with them.

04 · Handlers

Handlers return strings.

SRC/TOOLS.JS · *source*

```
const HANDLERS = {
  list_files(ws, args) { /* ... */ },
  read_file(ws, args) { /* ... */ },
  create_file(ws, args, ctx) { /* sandbox.check(SAVE) */ },
  write_file(ws, args, ctx) { /* sandbox.check(SAVE) */ },
  edit_file(ws, args, ctx) { /* sandbox.check(EDIT) */ },
  async run_command(ws, args, ctx) { /* policy + shell */ },
  remember(ws, args, ctx) { /* ctx.memory */ },
  recall(ws, args, ctx) { /* ctx.memory */ },
};
```

The return value is exactly what the model reads back. There is no separate result object to interpret.
On screen: Each handler receives the workspace, validated arguments, and the run context. Read tools use the workspace. Write tools use the sandbox. Memory tools use the memory store. Every handler returns a string. That string goes straight into a tool message.

05 · Command policy

Commands have their own guards.

SRC/TOOLS.JS · *source*

```
const HARD_DENY = [
  { re: /\brm\s+(-[a-z]*\s+)?-[a-z]*r[a-z]*f?\s+/, why: 'recursive delete of a root path' },
  { re: /\bdel\s+\/[a-z]*[sq]/i, why: 'recursive/quiet Windows delete' },
  { re: /\b(format|diskpart)\b/i, why: 'disk formatting' },
  { re: /git\s+push\s+(--force|-f)\b/i, why: 'force push' },
  { re: /\b(curl|wget)\b[^|]*\|\s*(ba)?sh/i, why: 'piping the internet into a shell' },
  { re: />\s*\/dev\/sd/i, why: 'raw disk write' },
];
```

Hard deny — refused before the policy or the prompt.

Ask — the y/N confirm callback decides.

Safe env — a short allow-list, never process.env.

Bounds — 6,000-char output cap and a 30s timeout.

> **On screen:** run_command is not a normal tool. It starts with a hard-deny list. Recursive deletes, disk formatting, force pushes and piping the internet into a shell are refused outright. Then policy decides: allow, ask, or deny. The child process gets a short env allow-list, not your key.

06 · Write tools

Every mutation takes the same gate.

SRC/TOOLS.JS · *source*

```
create_file(ws, args, ctx) {
  const sandbox = sandboxFor(ctx);
  const target = sandbox.check(ACTIONS.SAVE, args.path);
  if (fs.existsSync(target.abs)) {
    return 'ERROR: ' + target.rel + ' already exists - use edit_file or write_file';
  }
  const { bytes } = sandbox.write(target, args.content);
  return 'created ' + target.rel + ' (' + bytes + ' bytes)';
}
```

create_file, write_file and edit_file all call sandbox.check before touching disk. There is no second path.

remember A tool is a capability. The sandbox is the permission check in front of it.

On screen: create_file asks the sandbox for a SAVE target before it checks whether the file exists. The same gate covers write_file and edit_file. There is no bypass. A refusal comes back as a FORBIDDEN string, and the model can choose another path.

07 · Dispatch

The dispatcher never throws.

SRC/TOOLS.JS · *source*

```
export async function runTool(name, rawArgs, ctx) {
  if (!HANDLERS[name]) {
    return 'ERROR: unknown tool "' + name + '". Available: ' + Object.keys(HANDLERS).join(', ');
  }
  const { args, error } = validate(name, rawArgs);
  if (error) return error;

  try {
    return await HANDLERS[name](ctx.workspace, args, ctx);
  } catch (err) {
    if (err instanceof ForbiddenError) return 'FORBIDDEN: ' + err.message;
    if (err instanceof PathError) return 'ERROR: ' + err.message;
    return 'ERROR: ' + (err.code ?? 'failed') + ': ' + err.message;
  }
}
```

Every failure becomes an observation. The loop can always append a tool message and try another step. **On screen:** runTool is the dispatcher. Unknown tools get an error string listing what exists. Validation errors return immediately. Handler errors are caught and turned into observations. The loop never has to catch a tool exception to keep its own invariants.

08 · The bounds

Every capability has a bound.

maxOutput — 6,000 characters per command result.

timeoutMs — 30,000 ms per command.

denyPatterns — 8 hard-deny command patterns.

envKeys — 13 environment keys passed to the child.

memory caps — maxEntries, maxEntryChars, promptChars.

> Unbounded tools are unbounded risk. The limits are part of the tool design, not an afterthought.
> **On screen:** Output, time, command patterns, environment and memory all have explicit bounds. The same bounded-autonomy argument applies to tools as to the step budget. A capability without a bound is not a design. It is a future incident.

09 · Recap

Eight capabilities. One dispatcher.

• 8 tools

• 1 dispatcher

• 8 deny patterns

→

Schemas are prompt text. Write them for the model, not only for validation.

→

Validate first. Never call a handler with untrusted arguments.

→

Mutations use the sandbox. One gate, no bypass.

→

Errors are observations. runTool returns strings, never throws.

> **remember** Tools are the model's hands. The dispatcher is what keeps those hands accountable.
> **On screen:** Eight tools, one dispatcher, eight hard-deny patterns. Schemas guide the model, validation protects handlers, and the sandbox guards mutations. Every error becomes a string the model can learn from.